

**LAPORAN PROJECT AKHIR PEMROSESAN TEKS
IMPLEMENTASI MACHINE LEARNING UNTUK IDENTIFIKASI
KOMENTAR SARKASME PADA KASUS PERSELINGKUHAN**



Disusun Oleh:

Fitri Saidatur Hanifah	24031554035
Meida Rifa Alfiani	24031554139
Darista Wardhani	24031554146

Dosen Pengampu Mata Kuliah:

Riskyana Dewi Intan Puspitasari, M.Kom.
Ike Fitriyaningsih, M.Si

**PRODI S1 SAINS DATA
FAKULTAS MATEMATIKA & ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
SEMESTER GASAL 2025/2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan media sosial memberikan peluang bagi masyarakat untuk menyampaikan opini secara terbuka. Namun, kondisi tersebut turut memunculkan perilaku komunikasi yang tidak sehat, ditandai dengan munculnya komentar yang bersifat sarkastik dan provokatif. Fenomena ini semakin mengemuka ketika terjadi isu sosial yang menjadi pusat perhatian publik, seperti kasus perselingkuhan yang melibatkan figur terkenal, sehingga memicu gelombang respons negatif dari warganet.

Komentar sarkasme tidak hanya memperburuk kualitas interaksi digital, tetapi juga dapat menimbulkan dampak psikologis bagi individu yang menjadi sasaran. Oleh karena itu, dibutuhkan suatu sistem yang mampu melakukan identifikasi otomatis terhadap bentuk komunikasi negatif, khususnya komentar sarkas di antara komentar non-sarkas, guna mendukung upaya pengendalian konten yang merugikan di media sosial.

Dalam ranah Natural Language Processing (NLP), berbagai pendekatan representasi teks telah dikembangkan untuk mengubah kalimat menjadi vektor numerik agar dapat diproses oleh model machine learning. Pendekatan berbasis statistik seperti TF-IDF (Term Frequency-Inverse Document Frequency) banyak digunakan karena mampu memberikan bobot kata berdasarkan tingkat kepentingannya dalam dokumen. Sementara itu, pendekatan berbasis representasi semantik seperti Word2Vec dan FastText mampu menangkap makna kata berdasarkan konteks dan struktur sub-kata, sehingga lebih adaptif terhadap variasi bahasa informal yang sering digunakan pada komentar media sosial.

Untuk melakukan klasifikasi antara dua label non-sarkas (label 0) dan sarkas (label 1) penelitian ini menerapkan tiga algoritma machine learning, yaitu Logistic Regression, Support Vector Machine (SVM), dan Random Forest. Masing-masing algoritma diuji menggunakan tiga teknik representasi teks (TF-IDF, Word2Vec, dan FastText), sehingga menghasilkan total sembilan model. Perbandingan performa dari sembilan model tersebut dilakukan untuk mengidentifikasi kombinasi representasi dan algoritma yang paling efektif dalam mendeteksi komentar sarkasme.

Berdasarkan kondisi tersebut, project ini bertujuan untuk membangun dan menganalisis sembilan model klasifikasi berbasis kombinasi metode representasi teks dan algoritma machine learning dalam mengidentifikasi komentar non-sarkas dan sarkas pada kasus perselingkuhan. Hasil penelitian diharapkan dapat berkontribusi pada pengembangan sistem analisis teks berbahasa Indonesia yang lebih akurat dan responsif terhadap penyebaran konten negatif di media sosial.

1.2 Tujuan

- 1.2.1 Menganalisis karakteristik komentar sarkasme yang muncul pada isu perselingkuhan di Indonesia berdasarkan data yang diperoleh dari platform YouTube dan Tiktok.

- 1.2.2 Mengidentifikasi pola bahasa, struktur kalimat, dan kata kunci dominan yang membedakan komentar sarkas dan non-sarkas melalui teknik pemrosesan teks berbasis TF-IDF, Word2Vec, dan FastText embedding, untuk memahami bagaimana ciri-ciri sarkasme muncul dalam konteks bahasa informal di media sosial.
- 1.2.3 Membangun dan membandingkan performa sembilan model machine learning (kombinasi tiga teknik representasi teks TF-IDF, Word2Vec, dan FastText dengan tiga algoritma klasifikasi: Logistic Regression, Support Vector Machine (SVM), dan FastText Classifier) dalam mengidentifikasi komentar sarkasme, dengan dua label keluaran yaitu sarkas dan non-sarkas.
- 1.2.4 Mengevaluasi efektivitas masing-masing model menggunakan metrik akurasi, precision, recall, dan F1-score guna menentukan pendekatan representasi teks dan algoritma klasifikasi yang paling optimal untuk deteksi sarkasme pada komentar media sosial berbahasa Indonesia.
- 1.2.5 Hasil analisis diharapkan dapat memberikan kontribusi dalam upaya pencegahan perilaku komunikasi negatif di ruang digital dan membantu menciptakan lingkungan media sosial yang lebih sehat.

1.3 Manfaat

- 1.3.1 Memahami pola penggunaan bahasa sarkastik melalui analisis komentar warganet pada kasus perselingkuhan sehingga dapat menunjukkan sejauh mana penyebaran bahasa negatif mempengaruhi persepsi moral, dinamika sosial, dan sensitivitas publik terhadap isu perselingkuhan di media sosial.
- 1.3.2 Mendorong pihak pemerintah maupun lembaga sosial dalam menyusun strategi mitigasi komunikasi negatif dengan menyediakan bahan edukasi, panduan etika berkomunikasi, atau kampanye literasi digital sebagai upaya menciptakan interaksi yang lebih sehat di ruang online.
- 1.3.3 Menjadi referensi dalam upaya pengendalian cyberbullying di media sosial, melihat tingkat penyebarannya, serta bagaimana opini publik yang sarkastik dapat berkembang menjadi tindakan merugikan secara kolektif.

BAB II

PEMBAHASAN

2.1 Dataset

a. Testing

Dataset didapat dari hasil scraping pada 2 platform online yang banyak digunakan oleh masyarakat, yaitu Youtube dan Tiktok. Pada platform Youtube kami melakukan scraping pada sebuah video podcast yang membahas kasus perselingkuhan. Dengan memanfaatkan API public youtube yang kami dapat dari google console, kami berhasil mendapatkan dataset komentar dari keempat video dengan batas maksimal komentar 1000 agar komentar dalam video tersebut terproses dengan maksimal. Berikut code inti yang kami gunakan untuk melakukan scraping komentar youtube:

```
from googleapiclient.discovery import build
import pandas as pd

API_KEY = "AIzaSyA34hqYuP2enwK_C0ouat6YMXNI_oMyjp4"
VIDEO_IDS = ["Srj0EO9N3X4"]

MAX_COMMENTS_WANTED = 1000

youtube = build('youtube', 'v3', developerKey=API_KEY)
```

Setelah berhasil mendapatkan 1000 komentar dari youtube, kami melakukan scraping 1000 komentar pada platform Tiktok. Scraping dilakukan dengan bantuan website Apify Store yang dapat diakses secara gratis. Apify Store akan membuat code python yang telah disesuaikan dengan link video tiktok yang akan discraping dengan menyesuaikan jumlah komentar yang dibutuhkan. Code yang dibuat oleh Apify Store dapat dijalankan dalam file colab, hasil akhir nya berupa DataFrame yang dapat dikonversi menjadi format file CSV. Berikut script yang dijalankan untuk melakukan scraping Tiktok:

```

from apify_client import ApifyClient

client = ApifyClient("apify_api_b5krwiQitG99k9ekHokZoQhx9yn6to207zNd")

run_input = {
    "commentsPerPost": 1000,
    "excludePinnedPosts": False,
    "maxRepliesPerComment": 0,
    "postURLs": [
        "https://www.tiktok.com/@yourbaee25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsiLQq"
    ],
    "resultsPerPage": 100
}

run = client.actor("BDec00yAmCm1QbMEI").call(run_input=run_input)
items = list(client.dataset(run["defaultDatasetId"]).iterate_items())

```

Melalui scraping Tiktok, peneliti memperoleh informasi penting seperti text, diggCount, likedByAuthor, pinnedByAuthor, repliesToId, replyCommentTotal, uid, uniqueId sebagai penunjang lancarnya prediksi ini. Sementara itu, scraping YouTube digunakan untuk mengumpulkan data seperti judul video, tanggal unggah, serta komentar pengguna. Kedua sumber data ini kemudian digabungkan dan disusun dalam format terstruktur (misalnya CSV/DataFrame) sehingga dapat diolah pada tahap berikutnya, seperti pembersihan data, eksplorasi, dan pemodelan sesuai tujuan penelitian. Berikut cuplikan code untuk melakukan integrasi dataset dari hasil sccrapping youtube dan Tiktok:

```

import pandas as pd

df_x = pd.read_csv('/content/drive/Shared drives/hasil scraping/tiktok_comments.csv')
df_ytb = pd.read_csv('ytb_comments.csv')

df_x["platform"] = "tiktok"
df_ytb["platform"] = "youtube"

df_combined = pd.concat([df_x, df_ytb], ignore_index=True)
df_combined.to_csv('combined_dataset.csv', index=False)

print("File berhasil disimpan sebagai combined_dataset.csv")

```

Dari perintah tersebut akan menghasilkan satu file csv yang sudah mengintegrasikan kedua file scraping. Integrasi berdasarkan kolom platform, berikut value csv yang terbentuk:

text	diggCount	likedByAuthor	pinnedByAuthor	repliesTold	replyCommentTotal	uid	uniqueId
	100656.0	True	False		842.0	7.370688019797984e+18	chntk__
	33891.0	False	False		82.0	7.502438956907791e+18	cocounute7
	20414.0	False	False		163.0	7.544876926142793e+18	bunga.rinjani747
saran teks	7413.0	False	False		94.0	7.564116728012735e+18	lazira_dharma
tebak perasaan daehoon pas jule ngmong bgitu	35157.0	False	False		91.0	7.220670033428678e+18	punya_oikawa
Maaf gugup 🙏	1063.0	False	False		9.0	7.162190687147001e+18	ittarsta0
Kok klo ngmong mulutnya tetep...	17910.0	False	False		34.0	7.484927742913693e+18	nurq29

b. Training

Melakukan deteksi komentar sarkas dan toxic memerlukan dataset untuk melatih model agar dapat dilakukan testing pada data scraping yang telah dimiliki. Dataset training berupa file CSV yang berisi kalimat yang telah dilabeli atau dikelompokkan ke dalam beberapa kategori. Pada kasus ini kami menggunakan dua dataset berikut:

- Dataset sarkas.csv

sarkas.csv	
A	B
tweet	label
<username> <userna	0
Beneran pengen tau	0
Anda anggota DPRD	0
<username> <userna	0
Kamu tau tidak.? Pa	0
Anies Janji Izinkan R	0
Terimakasih atas per	1
Stlh Kerajaan Keratr	1
Horeeee.....?????	1
Partai Allah mah Bel	1
Ssstttt liatin aja sam	0
Berhentilah jadi <ha	0
menang kami bangg	0

Dataset diunduh secara manual dari laman huggingface dalam format .csv Pada dataset sarkas.csv terdapat dua kolom -tweet dan label- dan 1879 baris. File tersebut merupakan salah satu data yang dipelajari model untuk mendeteksi komentar sarkas dan toxic pada file testing. Kalimat yang memiliki label 0 memiliki arti Non-sarkas dan kalimat dengan label 1 memiliki arti Sarkas.

2.2 Cleaning Data

Dataset hasil scraping yang telah terintegrasi dilakukan cleaning untuk menghapus duplikat, baris kosong, URL, mention (@), hastag (#), karakter aneh, dan spasi berlebih. Cleaning ini dilakukan dengan tujuan pada saat proses modelling output yang dikeluarkan mudah dipahami oleh semua kalangan dan menampilkan hasil yang lebih rapi. Berikut cuplikan code yang digunakan untuk melakukan cleaning:

```
df = pd.read_csv("combined_dataset.csv")
df = df.drop_duplicates()

text_col = "text"
df = df.dropna(subset=[text_col])

df[text_col] = df[text_col].str.replace(r"http\S+|www.\S+", "", regex=True)

df[text_col] = df[text_col].str.replace(r"@w+", "", regex=True)

df[text_col] = df[text_col].str.replace(r"#w+", "", regex=True)

df[text_col] = df[text_col].str.replace(r"\bRT\b", "", regex=True)

df[text_col] = df[text_col].str.replace(r"^[A-Za-z0-9\s!?,;()'\-]", " ", regex=True)

df[text_col] = df[text_col].str.replace(r"\s+", " ", regex=True).str.strip()

df.to_csv("cleaned_dataset.csv", index=False)

print("Cleaning selesai → file tersimpan: cleaned_dataset.csv")
```

Dari perintah tersebut akan menghasilkan dataset yang sudah di cleaning dan disimpan dalam file csv baru. Isi file cleaned_dataset.csv yang terbentuk memiliki value sebagai berikut:

11610 of 1947 entries									Filter	
	input	cid	createTime	createTimeISO	text	diggCount	likedByAuthor	pinned		
1?	https://www.tiktok.com/@yourbaae25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsILQq	7.571754082751972e+18	1762936382.0	2025-11-12T08:33:02.000Z	saran teks	7413.0	False	False		
1?	https://www.tiktok.com/@yourbaae25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsILQq	7.571047485134718e+18	1762771867.0	2025-11-10T10:51:07.000Z	tebak perasaan daehoon pas jule ngmong bgitu	35157.0	False	False		
1?	https://www.tiktok.com/@yourbaae25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsILQq	7.57148320094236e+18	1762873312.0	2025-11-11T15:01:52.000Z	Maaf gugup	1063.0	False	False		
1?	https://www.tiktok.com/@yourbaae25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsILQq	7.571225162542269e+18	1762813260.0	2025-11-10T22:21:00.000Z	Kok klo ngmong mulutnya kek gitu sih	17910.0	False	False		
1?	https://www.tiktok.com/@yourbaae25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsILQq	7.57182029134989e+18	1762951797.0	2025-11-12T12:49:57.000Z	kok dia klo ngmong mulutnya miring? ya knp ya??	70047.0	False	False		
1?	https://www.tiktok.com/@yourbaae25/video/7570254257987980551?_r=1&_t=ZS-91yEwgsILQq	7.571416769803224e+18	1762857848.0	2025-11-11T10:44:08.000Z	DITUDUH ITU	24905.0	False	False		

2.3 Pre-processing

Untuk melanjutkan ke proses analisis yang terstruktur diperlukan pre-processing. Tahap pre-processing pada project ini dilakukan untuk menyiapkan teks agar lebih bersih, terstandarisasi, dan dapat diproses secara optimal oleh model machine learning. Proses ini meliputi tujuh operasi utama, yaitu:

1. Pembersihan elemen non-informatif, seperti penghapusan URL, mention (@username), dan hashtag menggunakan pola regex agar teks yang dianalisis fokus pada isi komentar.
2. Normalisasi slang, yaitu mengganti kata-kata tidak baku (misalnya gk, bgt, pls, wkwk) menjadi padanan kata formal menggunakan slang dictionary yang telah ditentukan.
3. Konversi emoji ke representasi teks, dengan mengubah setiap emoji menjadi token deskriptif seperti emoji_smiling_face, sehingga makna emosional dalam komentar tetap dipertahankan sebagai fitur yang dapat diproses model.
4. Pemecahan emoji Unicode dari huruf, sehingga emoji yang menempel pada kata dapat dipisahkan dan dibaca sebagai token tersendiri.
5. Pengubahan semua teks menjadi lowercase untuk menjaga konsistensi representasi.
6. Penghapusan karakter non-alfanumerik yang tidak diperlukan, menjaga agar hanya huruf, angka, underscore, dan token emoji yang tersisa.
7. Perapian spasi berlebih, memastikan struktur kalimat bersih dan tidak mengandung whitespace yang berlebihan.

```
def preprocess(text):
    if not isinstance(text, str):
        return ""

    text = text.lower()

    text = re.sub(r"http\S+|www.\S+", " ", text)
    text = re.sub(r"@w+", " ", text)
    text = re.sub(r"#w+", " ", text)

    # Pisahkan semua huruf/angka/underscore dari emoji Unicode
    text = re.sub(r"([a-zA-Z0-9_])([\U0001F000-\U0001FAFF])", r"\1 \2", text)
    text = re.sub(r"([\U0001F000-\U0001FAFF])([a-zA-Z0-9_])", r"\1 \2", text)

    # convert emoji → text
    text = convert_emoji(text)

    text = normalize_slang(text)

    text = re.sub(r"^[a-zA-Z0-9_]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    return text
```



```
def preprocess_and_save(input_csv, text_column="text", output_csv="preprocessed_dataset.csv"):
    df = pd.read_csv(input_csv)

    print("Mulai preprocessing...")
    df["clean_text"] = df[text_column].apply(preprocess)

    df.to_csv(output_csv, index=False, encoding="utf-8-sig")
    print(f"Preprocessing selesai → file tersimpan: {output_csv}")

    return df
df = preprocess_and_save("cleaned_dataset.csv", text_column="text")
df.head()
```

Seluruh langkah ini menghasilkan teks yang telah dibersihkan dan terstandarisasi dalam kolom `clean_text`, yang kemudian disimpan dalam file CSV sebagai data siap pakai untuk proses ekstraksi fitur dan pemodelan pada tahap berikutnya.

id	mentions	detailedMentions	platform	videoid	commentId	author	publishedAt	likeCount	clean_text
2f0ffec00a3824966d9e4e512f~tplv-tiktokx- id=1765033200&x- ps=13740610&shp=30310797&shcp=ff37627b&idc=useast5	0	0	tiktok						saran teks
05ad901397c939515f9923182~tplv-tiktokx- id=1765033200&x- ps=13740610&shp=30310797&shcp=ff37627b&idc=useast5	0	0	tiktok						tebak perasaan daehoon pas jule ngmong bgitu
17a60bf15d8e992351a256a6bb~tplv-tiktokx- id=1765033200&x- ps=13740610&shp=30310797&shcp=ff37627b&idc=useast5	0	0	tiktok						maaf gugup moal
75307a79aadd37952d262b7127~tplv-tiktokx- id=1765033200&x- ps=13740610&shp=30310797&shcp=ff37627b&idc=useast5	0	0	tiktok						kok klo ngmong mulutnya kek gitu sih
b0d05da5239c638aca42cc76f4~tplv-tiktokx- id=1765033200&x- ps=13740610&shp=30310797&shcp=ff37627b&idc=useast5	0	0	tiktok						kok dia klo ngomong mulutnya miring ya knp emoji_ya thinking_facethinking_face

2.4 Exploratory Data Analyst

Exploratory Data Analysis (EDA) adalah tahap awal analisis data yang dilakukan untuk memahami karakteristik data, memeriksa kualitasnya, serta mengidentifikasi pola, distribusi, dan potensi masalah seperti missing value atau outlier. Pada tahap ini, peneliti melakukan rangkaian analisis deskriptif dan visualisasi agar mendapatkan gambaran awal mengenai struktur dan isi dataset sebelum masuk ke tahap pemodelan lanjutan.

Tujuan Exploratory Data Analysis (EDA) adalah untuk memahami terlebih dahulu ukuran dan struktur dataset yang digunakan, meliputi jumlah baris, jumlah kolom, serta tipe data di setiap variabel. Selain itu, EDA ini juga bertujuan untuk mengecek kualitas data, khususnya terkait keberadaan missing value dan letaknya di setiap kolom, serta melihat karakteristik statistik data numerik seperti nilai minimum, maksimum, rata-rata, dan sebarannya.

Di sisi lain, EDA ini dilakukan untuk memahami distribusi teks dan variabel-variabel khusus yang relevan dengan penelitian. Hal ini mencakup analisis panjang teks (jumlah karakter), jumlah kata pada setiap teks, distribusi variabel `favorite_count` (misalnya jumlah like atau retweet), serta komposisi data berdasarkan platform (YouTube dan Tiktok). Selain itu, kami juga ingin mengetahui seberapa banyak teks yang mengandung kata kunci tertentu terkait perselingkuhan (misalnya “selingkuh”, “affair”, “cheat”, dan “perselingkuhan”) untuk melihat proporsi data yang benar-benar berkaitan langsung dengan topik penelitian.

- Load data & informasi struktur data

```

Jumlah baris: 1947
Jumlah kolom: 24

=== INFO DATA ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1947 entries, 0 to 1946
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   videoWebUrl            947 non-null    object
1   submittedVideoUrl      947 non-null    object
2   input                  947 non-null    object
3   cid                    947 non-null    float64
4   createTime             947 non-null    float64
5   createTimeISO          947 non-null    object
6   text                   1947 non-null   object
7   diggCount              947 non-null    float64
8   likedByAuthor          947 non-null    object
9   pinnedByAuthor         947 non-null    object
10  repliesToId            0 non-null      float64
11  replyCommentTotal      947 non-null    float64
12  uid                    947 non-null    float64
13  uniqueId               947 non-null    object
14  avatarThumbnail        947 non-null    object
15  mentions               947 non-null    object
16  detailedMentions       947 non-null    object
17  platform               1947 non-null   object
18  videoId                1000 non-null   object
19  commentId              1000 non-null   object
20  author                 1000 non-null   object
21  publishedAt            1000 non-null   object
22  likeCount              1000 non-null   float64
23  clean_text             1938 non-null   object
dtypes: float64(7), object(17)
memory usage: 365.2+ KB
None

```

Pada tahap awal, membaca dataset gabungan dan mengecek jumlah baris dan kolom. Hal ini dilakukan untuk mengetahui ukuran dataset dan memastikan data yang dibaca sudah sesuai dengan yang diharapkan. Analisis struktur data dilakukan menggunakan fungsi `info()` dan `dtypes` untuk melihat tipe data setiap kolom serta jumlah nilai yang terisi. Langkah ini penting untuk mendeteksi adanya kolom dengan banyak nilai kosong (null) dan memastikan tipe data sudah sesuai untuk analisis selanjutnya.

```

=== TIPE DATA ===
videoWebUrl      object
submittedVideoUrl object
input            object
cid              float64
createTime       float64
createTimeISO    object
text             object
diggCount        float64
likedByAuthor    object
pinnedByAuthor   object
repliesToId      float64
replyCommentTotal float64
uid              float64
uniqueId         object
avatarThumbnail  object
mentions         object
detailedMentions object
platform         object
videoId          object
commentId        object
author           object
publishedAt      object
likeCount        float64
clean_text       object
dtype: object

=== DESKRIPSI NUMERIK ===

```

	cid	createTime	diggCount	repliesToId	\
count	9.470000e+02	9.470000e+02	947.000000	0.0	
mean	7.575486e+18	1.763805e+09	203.111932	NaN	
std	2.805748e+15	6.532605e+05	2777.923555	NaN	
min	7.570350e+18	1.762610e+09	0.000000	NaN	
25%	7.573067e+18	1.763242e+09	1.000000	NaN	
50%	7.575361e+18	1.763776e+09	2.000000	NaN	
75%	7.577759e+18	1.764335e+09	3.000000	NaN	
max	7.580393e+18	1.764948e+09	70047.000000	NaN	

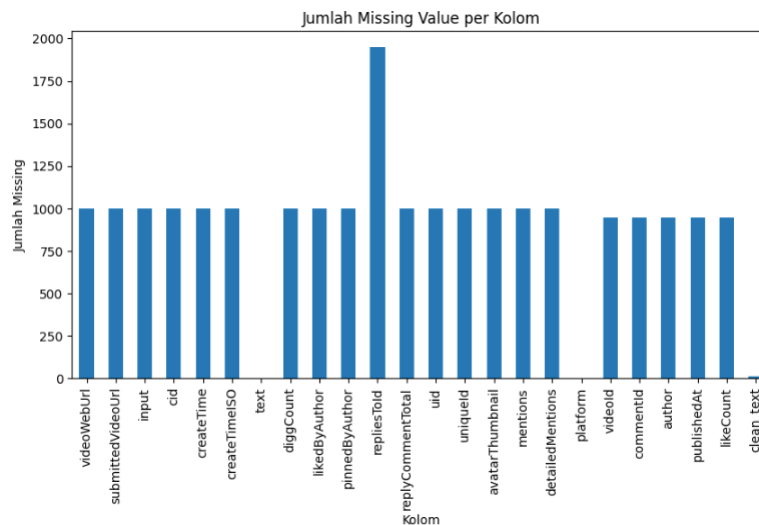
Tahap selanjutnya adalah melakukan deskripsi statistik terhadap variabel numerik menggunakan `describe()`. Statistik seperti nilai minimum, maksimum, rata-rata, dan kuartil membantu kami memahami sebaran data, mendeteksi kemungkinan nilai ekstrem (outlier), serta melihat skala masing-masing variabel.

- Missing values & Distribusi Panjang Teks

```

=== JUMLAH MISSING VALUE PER KOLOM ===
videoWebUrl      1000
submittedVideoUrl 1000
input            1000
cid              1000
createTime       1000
createTimeISO    1000
text             0
diggCount        1000
likedByAuthor    1000
pinnedByAuthor   1000
repliesToId      1947
replyCommentTotal 1000
uid              1000
uniqueId         1000
avatarThumbnail  1000
mentions         1000
detailedMentions 1000
platform         0
videoId          947
commentId        947
author           947
publishedAt      947
likeCount        947
clean_text       9
dtype: int64

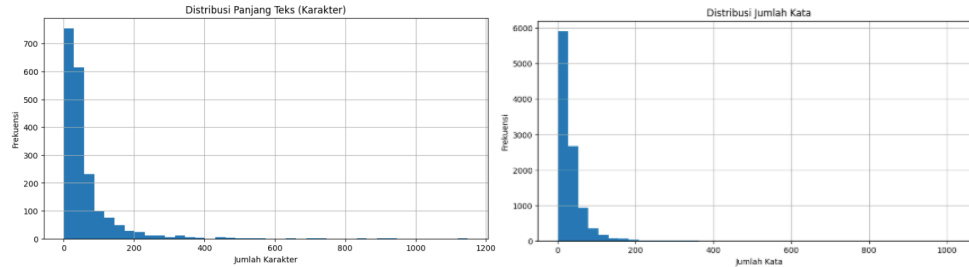
```



df.isna().sum() berfungsi untuk menghitung jumlah nilai hilang (NaN) di setiap kolom dan memvisualisasi kolom mana yang banyak missing value dengan diagram bar plot. Cek kualitas data dengan menghitung jumlah missing value di setiap kolom. Selain disajikan dalam bentuk tabel, informasi ini juga divisualisasikan menggunakan diagram

batang agar terlihat dengan jelas kolom mana yang memiliki proporsi data hilang paling besar. Temuan ini menjadi dasar dalam menentukan strategi penanganan missing value pada tahap preprocessing.

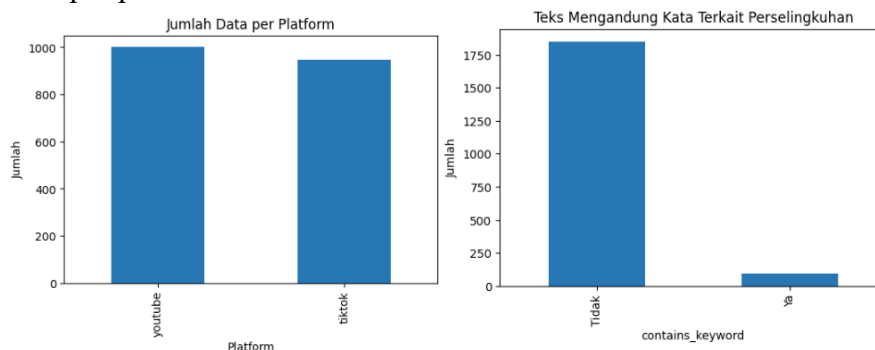
- Distribusi Jumlah Kata & Panjang Kata



Untuk memahami karakteristik konten teks, kami membuat fitur turunan berupa panjang teks (`text_length`) yang dihitung dari jumlah karakter pada kolom `full_text`. Histogram distribusi panjang teks digunakan untuk melihat apakah teks cenderung pendek, sedang, atau panjang, serta mendeteksi adanya teks yang terlalu pendek (mungkin tidak informatif) atau sangat panjang.

Selain panjang karakter, kami juga menghitung jumlah kata (`word_count`) untuk setiap teks. Distribusi jumlah kata memberikan gambaran tambahan mengenai kompleksitas dan kelengkapan teks, misalnya apakah mayoritas postingan hanya terdiri dari sedikit kata atau berupa paragraf yang lebih panjang. Jika tersedia, variabel `favorite_count` dianalisis untuk melihat distribusi tingkat interaksi pengguna (misalnya jumlah like pada postingan). Visualisasi ini membantu memahami apakah data interaksi cenderung terkonsentrasi pada nilai yang rendah (banyak postingan dengan sedikit like) atau terdapat sejumlah kecil postingan yang sangat populer.

- Distribusi data per platform & frekuensi kemunculan kata kunci



Kami melihat komposisi data berdasarkan sumber atau platform menggunakan variabel `platform`. Diagram batang jumlah data per platform menunjukkan apakah data lebih banyak berasal dari YouTube dan TikTok, serta membantu mengevaluasi keseimbangan sumber data dalam dataset gabungan.

Selanjutnya, kami melakukan eksplorasi berbasis konten dengan menandai teks yang mengandung kata kunci tertentu, seperti “selingkuh”, “affair”, “cheat”, dan

“perselingkuhan”. Hasilnya disimpan dalam variabel baru `contains_keyword`. Visualisasi jumlah teks yang mengandung kata kunci dibandingkan yang tidak, memberikan gambaran awal mengenai proporsi data yang relevan langsung dengan topik perselingkuhan dalam keseluruhan dataset.

- Statistik Deskriptif

```
=== STATISTIK DESKRIPTIF (text_length, word_count, favorite_count) ===
      text_length  word_count
count  1947.000000  1947.000000
mean    62.198767   10.428351
std     84.866639   13.906137
min      1.000000    1.000000
25%     21.000000    4.000000
50%     37.000000    6.000000
75%     67.000000   11.000000
max    1148.000000  183.000000
```

Analisis dilanjutkan dengan menghitung statistik deskriptif untuk variabel `text_length`, `word_count`, dan `favorite_count`. Langkah ini bertujuan untuk memahami sebaran nilai, rata-rata, serta rentang (minimum–maksimum) dari ketiga variabel tersebut sehingga diperoleh gambaran awal mengenai karakteristik panjang teks dan tingkat interaksi pengguna.

2.5 Modelling

Pada tahap modelling ini, berbagai algoritma machine learning diterapkan untuk mengevaluasi kemampuan sistem dalam mengklasifikasikan teks sarkasme. Setelah data melalui proses pembersihan, augmentasi, serta konversi ke representasi fitur seperti TF-IDF, Word2Vec, dan FastText setiap model kemudian dilatih dan diuji untuk melihat performanya dalam mengenali pola bahasa yang bersifat sarkastik. Hasil evaluasi dari masing-masing model memberikan gambaran mengenai akurasi, ketepatan prediksi, serta kemampuan generalisasi, sehingga dapat ditentukan pendekatan mana yang paling efektif digunakan pada tugas klasifikasi ini.

2.5.1 TF IDF

TF-IDF (Term Frequency – Inverse Document Frequency) yaitu untuk mengubah teks mentah menjadi representasi numerik yang dapat diproses oleh model machine learning. TF-IDF bekerja dengan menilai seberapa penting suatu kata dalam sebuah dokumen dibandingkan seluruh dokumen lain dalam dataset.

```

synonyms = {
    "bagus": ["keren", "mantap", "luar biasa"],
    "hebat": ["luar biasa", "mantap benar", "keren sekali"],
    "pintar": ["cerdas sekali", "hebat benar", "jago kali"],
    "wah": ["wow", "cie", "ciee"],
    "ternyata": ["rupanya", "nyatanya"],
    "sekali": ["banget", "amat sangat"],
    "selingkuh": ["main belakang", "cheating", "main gila"],
    "pelakor": ["orang ketiga", "perebut suami orang"],
    "pengkhianat": ["tidak setia", "penipu perasaan"],
    "bohong": ["ngibul", "tidak jujur"],
    "pasangan": ["partner", "doi", "gebetan"],
    "goblok": ["bodoh", "tolol"],
    "kampret": ["bangsat", "brengsek"],
    "kesel": ["jengkel", "sebel"],
    "marah": ["geram", "emosi"],
    "capek": ["lelah", "tired"],
}

def synonym_replace(sentence):
    words = sentence.split()
    new_words = []
    for w in words:
        wl = w.lower()
        if wl in synonyms:
            new_words.append(random.choice(synonyms[wl]))
        else:
            new_words.append(w)
    return " ".join(new_words)

```

Pada kode ini digunakan teknik *synonym replacement* untuk memperbanyak variasi kalimat tanpa mengubah makna dasarnya. Fungsi `synonym_replace()` bekerja dengan memecah kalimat input menjadi kata-kata terpisah, kemudian memeriksa setiap kata apakah ada dalam daftar sinonim. Jika kata tersebut ditemukan dalam kamus, fungsi akan menggantinya dengan salah satu sinonim secara acak menggunakan `random.choice()`. Jika tidak, kata tersebut dibiarkan tetap. Setelah seluruh kata diproses maka hasil akan digabungkan kembali menjadi sebuah kalimat baru. Dengan metode ini, dataset dapat diperluas secara otomatis, menghasilkan variasi kalimat yang lebih banyak sehingga model dapat belajar pola bahasa yang lebih beragam.

```

def random_swap(sentence):
    words = sentence.split()
    if len(words) < 2:
        return sentence
    i, j = random.sample(range(len(words)), 2)
    words[i], words[j] = words[j], words[i]
    return " ".join(words)

```

Random swap yaitu metode yang bekerja dengan menukar posisi dua kata secara acak dalam sebuah kalimat. Fungsi `random_swap()` untuk memecah kalimat menjadi daftar kata. Jika jumlah kata kurang dari dua, fungsi langsung mengembalikan kalimat asli karena tidak ada kata yang bisa ditukar. Namun, jika jumlah kata mencukupi, fungsi memilih dua indeks berbeda secara acak menggunakan `random.sample()`, kemudian menukar posisi kata pada kedua indeks dan semua kata akan disusun kembali menjadi sebuah kalimat baru. Tujuan dari teknik ini adalah menambah variasi struktur kalimat sehingga model dapat belajar memahami konteks yang lebih beragam dan menjadi lebih beragam terhadap perubahan susunan kata dalam data nyata.

```

minority_label = df[label_col].value_counts().idxmin()
df_minority = df[df[label_col] == minority_label]

target = df[label_col].value_counts().max()
aug_rows = []

while len(df_minority) + len(aug_rows) < target:
    for text in df_minority[text_col]:
        aug_rows.append([synonym_replace(text), minority_label])
        aug_rows.append([random_swap(text), minority_label])

    if len(df_minority) + len(aug_rows) >= target:
        break

df_aug = pd.DataFrame(aug_rows, columns=[text_col, label_col])
df_final = pd.concat([df, df_aug], ignore_index=True)

```

Sistem mendeteksi kelas dengan jumlah data paling sedikit (kelas minoritas) menggunakan fungsi `value_counts().idxmin()`. Selanjutnya, kode menentukan jumlah target augmentasi, yaitu menyamakan jumlah data kelas minoritas dengan kelas mayoritas agar distribusi data menjadi seimbang. Untuk menambah jumlah data, digunakan dua teknik augmentasi: *synonym replacement*, yaitu mengganti kata tertentu dengan sinonim yang telah didefinisikan, serta *random swap*, yaitu menukar posisi dua kata dalam kalimat secara acak. Kedua teknik ini diaplikasikan secara bergantian pada seluruh data minoritas hingga jumlahnya mencapai jumlah kelas mayoritas. Dengan proses ini, dataset menjadi lebih seimbang sehingga model dapat belajar secara lebih optimal tanpa bias terhadap kelas tertentu.

```

Jumlah awal per label:
label
0    1408
1     470
Name: count, dtype: int64

Jumlah setelah augmentasi:
label
0    1408
1    1408
Name: count, dtype: int64

TF-IDF selesai & stabil:
- tfidf_train.csv
- tfidf_test.csv
- tfidf_vectorizer.pkl

```

Sebelum augmentasi, dataset memiliki distribusi yang tidak seimbang, yaitu label 0 sebanyak 1408 sampel, sedangkan label 1 hanya 470 sampel. Ketidakseimbangan ini berpotensi menyebabkan model cenderung bias terhadap kelas mayoritas. Oleh karena itu, dilakukan augmentasi khusus pada label minoritas. Setelah proses augmentasi menggunakan teknik seperti *synonym replacement* dan *random swap*, jumlah data pada label 1 berhasil ditingkatkan hingga mencapai 1408 sampel, sehingga kedua label memiliki jumlah yang sama. Dengan distribusi yang sudah seimbang, model dapat dilatih secara lebih stabil dan adil. Selanjutnya, proses ekstraksi fitur menggunakan TF-

IDF telah selesai dan menghasilkan tiga file, yaitu `tfidf_train.csv`, `tfidf_test.csv`, dan `tfidf_vectorizer.pkl`.

- TF IDF dengan Logistic Regression

```
train_df = pd.read_csv("tfidf_train.csv")
test_df = pd.read_csv("tfidf_test.csv")

X_train = train_df.drop(columns=["label"])
y_train = train_df["label"]

X_test = test_df.drop(columns=["label"])
y_test = test_df["label"]

logreg = LogisticRegression(
    max_iter=600,
    C=2.0,
    class_weight="balanced",
    random_state=SEED,
    solver="liblinear"
)

logreg.fit(X_train, y_train)
print("Model Logistic Regression selesai dilatih!")
```

Pada tahap ini proses pelatihan model menggunakan algoritma Logistic Regression. Data hasil ekstraksi fitur TF-IDF yang telah disimpan sebelumnya dimuat kembali melalui file `tfidf_train.csv` dan `tfidf_test.csv`. Pada tahap ini, fitur dipisahkan dari label sehingga data pelatihan terdiri atas matriks fitur (`X_train`) dan target kelas (`y_train`). Model Logistic Regression kemudian dikonfigurasi dengan parameter `max_iter` sebesar 600 agar proses optimasi dapat berjalan hingga konvergen, serta nilai `C = 2.0` untuk mengatur tingkat regularisasi. Selain itu, parameter `class_weight="balanced"` diterapkan untuk memastikan model memberikan bobot yang proporsional pada kedua kelas. Model dilatih menggunakan solver `liblinear` yang sesuai untuk dataset berukuran menengah dengan data sparse seperti hasil TF-IDF. Setelah proses pelatihan selesai, model Logistic Regression telah siap digunakan untuk evaluasi dan prediksi.

```

===== AKURASI =====
Accuracy: 0.9096

===== CLASSIFICATION REPORT =====

```

	precision	recall	f1-score	support
0	0.95	0.87	0.91	282
1	0.88	0.95	0.91	282
accuracy			0.91	564
macro avg	0.91	0.91	0.91	564
weighted avg	0.91	0.91	0.91	564

```

===== CONFUSION MATRIX =====
[[244  38]
 [ 13 269]]

File disimpan:
- logreg_sarkas.pkl
- model_tfidf_LR.csv

```

Hasil evaluasi menunjukkan bahwa model Logistic Regression yang dilatih menggunakan fitur TF-IDF mampu menghasilkan performa klasifikasi yang sangat baik. Model mencapai akurasi sebesar 90,96%, yang menunjukkan bahwa sebagian besar data uji berhasil diprediksi dengan benar. Berdasarkan *classification report*, kelas 0 memiliki precision sebesar 0.95 dan recall sebesar 0.87, sedangkan kelas 1 memperoleh precision sebesar 0.88 dan recall 0.95. Nilai f1-score untuk kedua kelas berada pada angka 0.91, menunjukkan keseimbangan antara kemampuan model dalam mengidentifikasi kelas positif serta menghindari kesalahan prediksi. Matriks kebingungan menunjukkan bahwa model memprediksi 244 data kelas 0 secara benar dan salah pada 38 data, sementara untuk kelas 1 model memprediksi 269 data secara benar dan hanya salah pada 13 data. Secara keseluruhan, hasil tersebut mengindikasikan bahwa model mampu melakukan klasifikasi teks sarkas dengan performa yang konsisten dan seimbang pada kedua kelas. ebutuhan deployment serta file model_tfidf_LR.csv sebagai dokumentasi hasil prediksi.

- TF IDF dengan SVM

```

train_df = pd.read_csv("tfidf_train.csv")
test_df = pd.read_csv("tfidf_test.csv")
X_train = train_df.drop(columns=["label"])
y_train = train_df["label"]
X_test = test_df.drop(columns=["label"])
y_test = test_df["label"]

model_svm = LinearSVC(
    C=1.0,
    class_weight="balanced",
    max_iter=5000,
    random_state=SEED
)

model_svm.fit(X_train, y_train)
print("Model SVM selesai dilatih!")

```

Pada tahap ini dilakukan pelatihan model klasifikasi menggunakan algoritma Support Vector Machine (SVM) dengan kernel linear melalui kelas *LinearSVC*. Dataset TF-IDF yang telah diproses sebelumnya dimuat kembali dari file *tfidf_train.csv* dan *tfidf_test.csv*, kemudian dipisahkan menjadi fitur (*X_train*, *X_test*) dan label (*y_train*, *y_test*). Model SVM dikonfigurasi dengan nilai parameter $C = 1.0$ sebagai pengatur tingkat regularisasi, untuk mencegah overfitting dan menjaga margin tetap optimal. Selain itu, parameter *class_weight="balanced"* digunakan agar model memberikan bobot proporsional pada kedua kelas, terutama karena data awal bersifat tidak seimbang. Batas iterasi ditingkatkan hingga 5000 untuk memastikan proses optimasi dapat mencapai konvergensi pada data TF-IDF yang memiliki dimensi besar. Setelah seluruh parameter diatur, model dilatih menggunakan data latih dan menghasilkan model klasifikasi berbasis SVM yang siap dievaluasi pada data uji.

```

Model SVM selesai dilatih!

===== AKURASI MODEL SVM =====
Accuracy: 0.9397

===== CLASSIFICATION REPORT =====

```

	precision	recall	f1-score	support
0	0.97	0.90	0.94	282
1	0.91	0.98	0.94	282
accuracy			0.94	564
macro avg	0.94	0.94	0.94	564
weighted avg	0.94	0.94	0.94	564

```

===== CONFUSION MATRIX =====
[[255  27]
 [  7 275]]

Prediksi test set disimpan ke model tfidf SVM.csv

```

Hasil evaluasi menunjukkan bahwa model Support Vector Machine (SVM) dengan kernel linear berhasil memberikan performa klasifikasi yang sangat baik pada data teks sarkas. Model mencapai akurasi sebesar 93,97%, yang merupakan peningkatan dibandingkan Logistic Regression. Berdasarkan *classification report*,

kelas 0 memiliki precision sebesar 0.97 dan recall sebesar 0.90, sedangkan kelas 1 memperoleh precision sebesar 0.91 dan recall 0.98. Kedua kelas menghasilkan nilai f1-score sebesar 0.94, menunjukkan keseimbangan yang kuat antara kemampuan mendeteksi kelas positif dan meminimalkan kesalahan prediksi. Matriks kebingungan menunjukkan bahwa model berhasil mengklasifikasikan 255 data kelas 0 dan 275 data kelas 1 secara benar, dengan jumlah kesalahan yang relatif kecil (27 kesalahan pada kelas 0 dan 7 kesalahan pada kelas 1). Secara keseluruhan, hasil ini mengindikasikan bahwa SVM mampu membangun batas keputusan yang sangat efektif untuk memisahkan teks sarkas dan non-sarkas.

- TF IDF dengan Random Forest

```
train_df = pd.read_csv("tfidf_train.csv")
test_df  = pd.read_csv("tfidf_test.csv")

X_train = train_df.drop(columns=["label"])
y_train = train_df["label"]

X_test  = test_df.drop(columns=["label"])
y_test  = test_df["label"]

rf = RandomForestClassifier(
    n_estimators=300,
    class_weight="balanced",
    random_state=SEED,
    n_jobs=1
)

rf.fit(X_train, y_train)
print("Model Random Forest selesai dilatih!")
```

Model Random Forest Classifier dilatih menggunakan data hasil ekstraksi fitur TF-IDF. Model Random Forest dibangun dengan 300 pohon keputusan ($n_estimators=300$), yang bertujuan untuk meningkatkan stabilitas prediksi dan mengurangi risiko overfitting. Parameter `class_weight="balanced"` digunakan untuk menyesuaikan bobot kelas secara otomatis berdasarkan frekuensi kemunculannya, sehingga model tidak bias terhadap kelas mayoritas. Selain itu, `random_state` diset dengan nilai konstan agar hasil pelatihan konsisten setiap kali kode dijalankan. Parameter `n_jobs=1` memastikan proses komputasi berjalan pada satu thread, sehingga lebih stabil pada lingkungan seperti Google Colab. Setelah konfigurasi selesai, model dilatih menggunakan perintah `rf.fit(X_train, y_train)` yang memungkinkan Random Forest mempelajari pola dan hubungan antar fitur untuk membedakan antara teks sarkas dan non-sarkas. Tahap pelatihan ini diakhiri

dengan pesan "Model Random Forest selesai dilatih!" sebagai indikator bahwa proses training berhasil berjalan tanpa error.

```
Model Random Forest selesai dilatih!

===== AKURASI MODEL RF =====
Accuracy: 0.9574

===== CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

         0           0.94       0.97       0.96         282
         1           0.97       0.94       0.96         282

   accuracy                   0.96         564
  macro avg           0.96       0.96       0.96         564
 weighted avg           0.96       0.96       0.96         564

===== CONFUSION MATRIX =====
[[274   8]
 [ 16 266]]

Prediksi RF disimpan ke model tfidf RF.csv
```

Hasil evaluasi menunjukkan bahwa model Random Forest yang dilatih menggunakan fitur TF-IDF memberikan performa yang sangat baik dalam mendeteksi teks sarkasme. Model ini mencapai akurasi sebesar 95.74%, yang berarti sebagian besar data uji berhasil diklasifikasikan dengan benar. Berdasarkan *classification report*, kelas 0 (non-sarkas) memiliki nilai precision sebesar 0.94 dan recall sebesar 0.97, sedangkan kelas 1 (sarkas) memperoleh precision sebesar 0.97 dan recall sebesar 0.94. Kedua kelas memiliki nilai f1-score yang sama, yaitu 0.96, menandakan performa yang seimbang dan konsisten antar kelas. Hasil ini diperkuat oleh *confusion matrix*, di mana model hanya membuat sedikit kesalahan: 8 data non-sarkas salah diklasifikasikan sebagai sarkas, dan 16 data sarkas salah diklasifikasikan sebagai non-sarkas. Performa tinggi ini menunjukkan bahwa Random Forest mampu menangkap pola teks dengan cukup baik berkat kombinasi banyak pohon keputusan yang bekerja secara ensemble.

2.5.2 Word2Vec

Word2Vec adalah sebuah teknik *word embedding* yang digunakan untuk mengubah kata menjadi representasi angka (vektor) sehingga bisa dipahami oleh model machine learning. Secara sederhana Word2Vec memetakan setiap kata ke dalam bentuk vektor berukuran tetap di mana kata-kata yang memiliki makna atau konteks yang mirip akan berada pada posisi yang berdekatan dalam ruang vektor tersebut.

Word2Vec akan melakukan tokenizing untuk memecah string berdasarkan spasi, namun karena sebelumnya sudah melakukan preprocessing yang sudah menghapus tanda baca, lowercasing, dll.

```
w2v_model = Word2Vec(
    sentences=tokenized_texts,
    vector_size=200,
    window=5,
    min_count=2,
    workers=8,
    sg=1,
    seed=42,
    epochs=20
)
```

Dilanjutkan dengan proses inti untuk melakukan training Word2Vec dimana setiap kata dipresentasi oleh vektor 200 dimensi, dengan konteks kiri-kanan sebanyak 5 kata yang dipakai untuk belajar. Kata yang muncul kurang dari 2 kali akan diabaikan untuk mengurangi noise pada proses prediksi, training Word2Vec menggunakan skip-gram untuk hasil lebih baik pada kata slang atau rare. Langkah akhir model yang sudah terlatih disimpan dalam file dan siap dipakai berulang kali.

Word2Vec training bersifat stokastik yang artinya pada setiap run akan memiliki hasil yang sedikit berbeda. Solusi dari masalah ini ialah dengan menambahkan seed dan epoch agar setiap run memiliki hasil yang sama dan konsisten. Word2Vec yang berhasil dibangun dilanjut untuk membuat embedding untuk dataset sarkas dengan script sebagai berikut:

```
def sentence_vector(tokens, model):
    vectors = [model.wv[t] for t in tokens if t in model.wv]
    if len(vectors) == 0:
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)

df_sarkas["tokens"] = df_sarkas["tweet"].astype(str).apply(tokenize)
df_sarkas["embedding"] = df_sarkas["tokens"].apply(lambda x: sentence_vector(x, w2v_model))

print("Embedding untuk dataset sarkas berhasil dibuat!")
```

Untuk setiap teks sarkas yang sudah di tokenisasi akan diambil vektor setiap kata dari model Word2Vec, jika ada dalam model maka akan dirata-rata kan semua vektor katanyajadi vektor kalimat (sentence embedding). Namun, jika tidak ada satupun kata di vocabulary, maka akan dikembalikan menjadi vektor nol. Dalam pembangunan model Word2Vec juga dilakukan augmentasi pada dataset sarkas untuk menyeimbangkan kelas minoritas supaya model tidak bias ke kelas mayoritas. Langkah akhirnya melakukan split train-test yang siap dipakai untuk melatih model dan evaluasi

Machine Learning. Berikut output yang dikeluarkan dari proses membangun model Word2Vec:

```
Jumlah scraping: 1947
Jumlah sarkas: 1878
Total teks gabungan: 3825
Contoh token: ['saran', 'teks']
Word2Vec selesai dilatih & disimpan!
Embedding untuk dataset sarkas berhasil dibuat!
Melakukan augmentasi kelas minoritas...

Jumlah label setelah augmentasi:
label
0      1408
1      1408
Name: count, dtype: int64

Train-test split selesai!
Ukuran X_train: (2252, 200)
Ukuran X_test: (564, 200)
```

- Word2Vec dengan Logistic Regression

Embedding Word2Vec sudah mengubah setiap kalimat menjadi vektor numerik berdimensi tetap, sehingga dapat digunakan oleh algoritma machine learning seperti Logistic Regression. Hal pertama yang dilakukan untuk melatih model *Logistic Regression* menggunakan fitur hasil embedding Word2Vec adalah inisialisasi dengan script sebagai berikut:

```
lr_model = LogisticRegression(
    max_iter=2000,
    class_weight="balanced",
    C=1.5,
    solver="lbfgs",
    random_state=42
)
```

Iterasi maksimal 2000 sebab data Word2Vec yang padat akan membantu model mencapai konvergensi dengan dibantu keseimbangan kelas minoritas agar tidak

bias. Sama seperti saat membangun model Word2Vec, pada melatih Logistic Regression juga dinyatakan `random_state = 42` Agar hasil model stabil dan dapat direproduksi setiap kali kode dijalankan.

```
y_pred = lr_model.predict(X_test)

print("\n==== AKURASI LR (W2V) =====")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))

print("\n==== CLASSIFICATION REPORT =====")
print(classification_report(y_test, y_pred))

print("\n==== CONFUSION MATRIX =====")
print(confusion_matrix(y_test, y_pred))
```

Setelah model dilatih menggunakan data training, model diuji menggunakan data test, lalu dihitung akurasi, precision, recall, f1-score, dan confusion matrix. Hasil prediksi kemudian disimpan ke file CSV untuk dokumentasi dan analisis lebih lanjut. Berikut output yang dihasilkan dari script tersebut:

```
==== AKURASI LR (W2V) =====
Accuracy: 0.7819

==== CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

     0           0.81       0.74       0.77        282
     1           0.76       0.82       0.79        282

   accuracy                   0.78        564
  macro avg                   0.78        564
weighted avg                   0.78        564

==== CONFUSION MATRIX =====
[[209  73]
 [ 50 232]]

Prediksi Logistic Regression disimpan ke model_W2V_LR.csv
```

- Word2Vec dengan SVM

Membangun dan mengevaluasi model Support Vector Machine (SVM) berbasis *Linear Support Vector Classifier* untuk tugas klasifikasi sarkasme. Fitur yang digunakan berasal dari embedding Word2Vec yang sudah diubah menjadi vektor numerik berdimensi tetap. Sama seperti sebelumnya, langkah pertama adalah menginisialisasi dengan script sebagai berikut:

```
svm_model = LinearSVC(  
    class_weight="balanced",  
    dual=False,  
    max_iter=5000,  
    random_state=42  
)
```

Ketika jumlah fitur lebih kecil dibanding jumlah sampel (seperti Word2Vec dengan 200 fitur), maka `dual=False` jauh lebih cepat dan stabil. Pemilihan parameter ini membuat training lebih efisien dan menghindari waktu komputasi yang lama. Selanjutnya melakukan prediksi dengan produksi dengan script berikut:

```
y_pred = svm_model.predict(X_test)  
  
print("\n==== AKURASI SVM (W2V) =====")  
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))  
  
print("\n==== CLASSIFICATION REPORT =====")  
print(classification_report(y_test, y_pred))  
  
print("\n==== CONFUSION MATRIX =====")  
print(confusion_matrix(y_test, y_pred))
```

Model SVM dilatih menggunakan embedding Word2Vec sebagai representasi fitur. LinearSVC dipilih karena efisien untuk data berdimensi rendah seperti Word2Vec. Model kemudian dievaluasi menggunakan akurasi, precision, recall, F1-score, dan confusion matrix. Hasil prediksi disimpan dalam file CSV untuk keperluan analisis lebih lanjut. Berikut output yang dihasilkan dari script tersebut:

```

===== AKURASI SVM (W2V) =====
Accuracy: 0.7926

===== CLASSIFICATION REPORT =====

```

	precision	recall	f1-score	support
0	0.82	0.75	0.78	282
1	0.77	0.84	0.80	282
accuracy			0.79	564
macro avg	0.79	0.79	0.79	564
weighted avg	0.79	0.79	0.79	564

```

===== CONFUSION MATRIX =====
[[211  71]
 [ 46 236]]

Prediksi SVM disimpan ke model_W2V_SVM.csv

```

- Word2Vec dengan Random Forest

Melatih dan mengevaluasi model Random Forest Classifier dengan fitur yang berasal dari embedding Word2Vec. Embedding ini sebelumnya telah mengubah teks menjadi vektor numerik sehingga dapat diproses oleh algoritma machine learning. Masih sama seperti sebelumnya, dilakukan insialisasi pada langkah pertama dengan script berikut:

```

rf_w2v = RandomForestClassifier(
    n_estimators=300,
    max_depth=25,
    min_samples_split=5,
    min_samples_leaf=2,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

```

Ketentuan inisialisasi kurang lebih sama dengan sebelumnya. Saat ini bisa dilanjutkan dengan melatih random forest dengan scriipt sebagai berikut:

```

y_pred_rf_w2v = rf_w2v.predict(X_test)

print("\n=== Random Forest (Word2Vec) ===")
print("Accuracy:", round(accuracy_score(y_test, y_pred_rf_w2v), 4))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_rf_w2v))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_rf_w2v))

```

Model Random Forest dilatih menggunakan fitur Word2Vec yang mengubah setiap kalimat menjadi vektor numerik berdimensi 200. Parameter seperti `max_depth`, `min_samples_split`, dan `min_samples_leaf` digunakan untuk mengontrol kompleksitas model agar tidak overfitting. Setelah model dilatih dengan 300 pohon keputusan, performanya diuji menggunakan data uji dan dievaluasi dengan akurasi, precision, recall, f1-score, dan confusion matrix. Hasil prediksi kemudian disimpan dalam file CSV untuk analisis lebih lanjut. Berikut output yang dihasilkan dari script tersebut:

```

=== Random Forest (Word2Vec) ===
Accuracy: 0.9379

Classification Report:

```

	precision	recall	f1-score	support
0	0.98	0.90	0.94	282
1	0.90	0.98	0.94	282
accuracy			0.94	564
macro avg	0.94	0.94	0.94	564
weighted avg	0.94	0.94	0.94	564

```

Confusion Matrix:
[[253  29]
 [  6 276]]

Prediksi RF Word2Vec disimpan ke model_W2V_RF.csv

```

2.5.3 Fast-Text

FastText memiliki kemampuan mempelajari subword (potongan kata), sehingga sangat sensitif terhadap keragaman bentuk kata. Karena itu augmentasi diterapkan pada

train set saja agar tidak terjadi kebocoran informasi, dan supaya variasi kata baru ikut menambah ragam subword yang dipelajari.

FastText dilatih secara deterministik dengan mengatur global seed pada semua komponen (Python, NumPy, dan FastText) sehingga seluruh proses mulai dari augmentasi, pelatihan FastText, hingga pembuatan embedding menghasilkan keluaran yang selalu sama setiap kali kode dijalankan.

```
SEED = 42

random.seed(SEED)
np.random.seed(SEED)

# Gensim FastText internal RNG
import gensim
gensim.models.word2vec.FAST_VERSION = 1 # percepat & stabil
```

Data dibagi terlebih dahulu menjadi train–test secara stratified agar test tetap “murni” dan tidak tersentuh augmentasi. Karena dataset awal tidak seimbang (label 1 jauh lebih sedikit), augmentasi dilakukan hanya pada train untuk menyeimbangkan distribusi dan menambah variasi kata yang akan dipelajari FastText tanpa menyebabkan data leakage. Hasilnya, FastText mempelajari subword dari semua kelas dengan lebih baik dan embedding yang dihasilkan menjadi lebih representatif serta stabil.

Berdasarkan kode, dataset sarkas awal memiliki ketidakseimbangan kelas yang cukup besar, yaitu label 0 sebanyak 1.408 data dan label 1 hanya 470 data. Setelah dilakukan train–test split dengan stratifikasi, ketidakseimbangan itu masih terlihat pada data train, yaitu 1126 data label 0 dan hanya 376 data label 1, sementara data test tetap tidak disentuh (282 label 0 dan 94 label 1).

```
Jumlah data awal per label:
label
0    1408
1     470
Name: count, dtype: int64

Label TRAIN sebelum augmentasi:
label
0    1126
1     376
Name: count, dtype: int64

Label TEST (TIDAK BOLEH DIAUGMENTASI):
label
0     282
1      94
Name: count, dtype: int64
```

Untuk mengatasi ketimpangan ini, kode menerapkan augmentasi hanya pada kelas minoritas di train set menggunakan dua strategi: synonym replacement (mengganti kata dengan sinonim dalam kamus) dan random swap (menukar posisi dua kata secara acak).

```
def synonym_replace(sentence):
    words = sentence.split()
    new_words = []
    for w in words:
        wl = w.lower()
        if wl in synonyms:
            # Random yang stabil karena SEED sudah dikunci
            new_words.append(random.choice(synonyms[wl]))
        else:
            new_words.append(w)
    return " ".join(new_words)

def random_swap(sentence):
    words = sentence.split()
    if len(words) < 2:
        return sentence

    # Pemilihan index stabil karena SEED sudah dikunci
    i, j = random.sample(range(len(words)), 2)
    words[i], words[j] = words[j], words[i]
    return " ".join(words)
```

Augmentasi diulang hingga jumlah kelas minoritas menyamai kelas mayoritas, menghasilkan 1126 data untuk kedua label. Proses ini penting karena memastikan bahwa model FastText akan dilatih dengan distribusi kelas yang seimbang tanpa menyebabkan kebocoran informasi pada data test.

```
minority_label = train_df["label"].value_counts().idxmin()
df_minority = train_df[train_df["label"] == minority_label]

target = train_df["label"].value_counts().max()
aug_rows = []
```

Setelah train set seimbang, kode melakukan tokenisasi dan melatih model FastText hanya pada data train hasil augmentasi, sehingga FastText belajar beragam pola kata dan subword yang lebih kaya. Model FastText diatur dengan mode skip-gram (sg=1) untuk menangkap konteks kata yang lebih luas pendekatan yang cocok untuk mendeteksi nuansa seperti sarkasme.

```
# 5. TOKENISASI
X_train_tokens = [text.split() for text in train_df["tweet"]]
X_test_tokens = [text.split() for text in test_df["tweet"]]

# 6. TRAIN FASTTEXT (STABIL)
fasttext_model = FastText(
    vector_size=100,
    window=5,
    min_count=2,
    workers=1,      # PENTING: workers=1 agar deterministik
    sg=1,           # PENTING: membuat training stabil
    seed=SEED
)

fasttext_model.build_vocab(X_train_tokens)

fasttext_model.train(
    X_train_tokens,
    total_examples=len(X_train_tokens),
    epochs=20
)

print("\nFastText selesai ditrain (hasil stabil 100%)")
```

Setelah pelatihan selesai, setiap kalimat pada train dan test dikonversi menjadi vektor 100 dimensi menggunakan rata-rata embedding kata (sentence embedding). Embedding inilah yang kemudian digunakan sebagai input untuk model klasifikasi.

Seluruh hasil embedding disimpan sebagai `fasttext_train_sarkas.csv` dan `fasttext_test_sarkas.csv`, sedangkan model FastText tersimpan sebagai `fasttext_model_sarkas.pkl`. Dengan demikian, alur kode memastikan bahwa representasi teks yang digunakan untuk klasifikasi berasal dari embedding yang stabil, kaya variasi, dan dilatih tanpa kontaminasi data uji.

```
train_ft_df = pd.DataFrame(X_train_ft)
train_ft_df["label"] = train_df["label"].values

test_ft_df = pd.DataFrame(X_test_ft)
test_ft_df["label"] = test_df["label"].values

train_ft_df.to_csv("fasttext_train_sarkas.csv", index=False)
test_ft_df.to_csv("fasttext_test_sarkas.csv", index=False)

print("\nFile tersimpan:")
print(" - fasttext_train_sarkas.csv")
print(" - fasttext_test_sarkas.csv")
```

```
✓ Label TRAIN setelah augmentasi:
label
0    1126
1    1126
Name: count, dtype: int64

✓ FastText selesai ditrain

✓ FASTTEXT selesai. File tersimpan:
- fasttext_train_sarkas.csv
- fasttext_test_sarkas.csv

✓ FastText model disimpan sebagai fasttext_model_sarkas.pkl
```

- Fast-Text dengan Logistic Regression

```
Jumlah data train: (2252, 100)
Jumlah data test : (376, 100)
✓ Model Logistic Regression (FastText) selesai dilatih!

===== AKURASI MODEL FASTTEXT =====
Accuracy: 0.7606

===== CLASSIFICATION REPORT FASTTEXT =====
              precision    recall  f1-score   support

0               0.85         0.82         0.84         282
1               0.52         0.57         0.55          94

 accuracy          0.76         0.76         0.76         376
 macro avg         0.69         0.70         0.69         376
 weighted avg      0.77         0.76         0.76         376

===== CONFUSION MATRIX FASTTEXT =====
[[232  50]
 [ 40  54]]

✓ Model disimpan sebagai logreg_fasttext_sarkas.pkl
✓ Prediksi test set disimpan ke model_predictions_fasttext_LR.csv
```

Berdasarkan hasil pelatihan dan evaluasi, model Logistic Regression dengan fitur embedding FastText menunjukkan performa yang cukup stabil dengan akurasi 76.06% pada data uji. Model mampu mengenali kelas mayoritas (label 0) dengan baik, ditunjukkan oleh nilai precision 0.85 dan recall 0.82, sedangkan performanya pada kelas minoritas (label 1) masih berada pada tingkat moderat dengan precision 0.52 dan recall 0.57. Hal ini menunjukkan bahwa meskipun model efektif dalam mengidentifikasi teks non-sarkas, kemampuannya dalam menangkap pola sarkasme yang lebih halus masih terbatas.

Confusion matrix memperlihatkan bahwa sebagian besar kesalahan prediksi disebabkan oleh salah klasifikasi pada kelas minoritas menjadi kelas mayoritas, yaitu sebanyak 50 prediksi salah pada label 0 dan 40 prediksi salah pada label 1. Kondisi ini mengindikasikan bahwa augmentasi yang dilakukan sebelumnya

memang membantu menyeimbangkan data latih, namun belum sepenuhnya mengatasi tantangan dalam mempelajari variasi linguistik khas sarkasme. Secara keseluruhan, kombinasi FastText dan Logistic Regression mampu memberikan generalisasi yang cukup baik, namun peningkatan lebih lanjut masih diperlukan, khususnya untuk memperbaiki representasi dan akurasi prediksi pada kelas minoritas.

- Fast Text dengan SVM

```
Jumlah data train: (2252, 100)
Jumlah data test : (376, 100)
✓ Model SVM (FastText) selesai dilatih!

===== AKURASI MODEL FASTTEXT + SVM =====
Accuracy: 0.7899

===== CLASSIFICATION REPORT FASTTEXT + SVM =====
              precision    recall  f1-score   support

     0           0.81       0.93       0.87         282
     1           0.64       0.36       0.46          94

   accuracy              0.79         376
  macro avg              0.73         376
 weighted avg              0.77         376

===== CONFUSION MATRIX FASTTEXT + SVM =====
[[263  19]
 [ 60  34]]

✓ Model disimpan sebagai svm_fasttext_sarkas.pkl
✓ Prediksi test set disimpan ke model_predictions_fasttext_SVM.csv
```

Berdasarkan hasil pelatihan dan evaluasi, model SVM yang menggunakan fitur embedding FastText menunjukkan performa yang cukup baik dengan akurasi 78.99% pada data uji. Model mampu mengenali kelas mayoritas (label 0) dengan sangat baik, tercermin dari nilai precision 0.81 dan recall 0.93 yang menunjukkan bahwa sebagian besar teks non-sarkas berhasil diprediksi secara benar dengan tingkat kesalahan yang rendah. Namun, performa pada kelas minoritas (label 1) masih berada pada level yang kurang optimal, dengan precision 0.64 dan recall 0.36. Hal ini mengindikasikan bahwa meskipun model efektif dalam membedakan teks non-sarkas, kemampuannya untuk memahami karakteristik bahasa sarkas yang lebih kontekstual dan implisit masih terbatas.

Confusion matrix menunjukkan pola kesalahan yang konsisten dengan metrik evaluasi, yaitu masih tingginya jumlah salah klasifikasi pada kelas minoritas. Tercatat 60 data berlabel sarkas diprediksi sebagai non-sarkas (false negative), sedangkan 19 data berlabel non-sarkas diprediksi sebagai sarkas (false positive). Kondisi ini menegaskan bahwa meskipun augmentasi data sebelumnya telah berhasil menyeimbangkan distribusi label pada data latih, keragaman pola linguistik khas sarkasme belum sepenuhnya terwakili dalam embedding yang dipelajari model. Secara keseluruhan, kombinasi FastText dan SVM mampu

memberikan generalisasi yang cukup stabil, namun peningkatan lebih lanjut terutama dalam memperkuat representasi kelas minoritas masih diperlukan agar model dapat lebih sensitif dalam mendeteksi pola sarkasme yang lebih subtil.

- TF IDF dengan Random Forest

```
Jumlah data train: (2252, 100)
Jumlah data test : (376, 100)
✓ Model Random Forest (FastText) selesai dilatih!

===== AKURASI MODEL FASTTEXT + RANDOM FOREST =====
Accuracy: 0.7846

===== CLASSIFICATION REPORT FASTTEXT + RANDOM FOREST =====
              precision    recall  f1-score   support

     0       0.81         0.94         0.87         282
     1       0.63         0.33         0.43          94

   accuracy          0.78         0.78         0.78         376
  macro avg       0.72         0.63         0.65         376
 weighted avg     0.76         0.78         0.76         376

===== CONFUSION MATRIX FASTTEXT + RANDOM FOREST =====
[[264  18]
 [ 63  31]]

✓ Model disimpan sebagai rf_fasttext_sarkas.pkl
✓ Prediksi test set disimpan ke model_predictions_fasttext_RF.csv
```

Berdasarkan hasil pelatihan dan evaluasi, model Random Forest yang menggunakan fitur embedding FastText menunjukkan performa yang cukup baik dengan akurasi 78.46% pada data uji. Model ini sangat kompeten dalam mengenali kelas mayoritas (label 0), ditunjukkan oleh nilai precision 0.81 dan recall 0.94 yang menegaskan bahwa sebagian besar teks non-sarkas berhasil diprediksi secara akurat. Namun, performa model pada kelas minoritas (label 1) masih berada pada tingkat yang rendah, dengan precision 0.63 dan recall hanya 0.33. Hal ini menunjukkan bahwa meskipun Random Forest mampu menangkap pola umum pada data mayoritas, kemampuannya dalam mendeteksi karakteristik sarkasme yang cenderung lebih halus dan kontekstual masih terbatas.

Confusion matrix menguatkan temuan tersebut, dengan terlihat bahwa model melakukan 63 kesalahan prediksi dengan mengklasifikasikan teks sarkas sebagai non-sarkas (false negative), dan 18 kesalahan pada arah sebaliknya (false positive). Pola ini menunjukkan bahwa augmentasi data memang membantu meningkatkan pembelajaran model, tetapi belum cukup efektif untuk memperkuat representasi kelas minoritas dalam embedding FastText. Secara keseluruhan, kombinasi FastText dan Random Forest menghasilkan generalisasi yang stabil dan konsisten, namun peningkatan lanjutan diperlukan untuk mengurangi bias terhadap kelas mayoritas dan meningkatkan sensitivitas model dalam mengenali variasi linguistik pada teks sarkasme.

2.6 Evaluasi Modelling

2.6.1 Membandingkan Seluruh Model

Modelling yang berhasil belajar dan dilakukan testing pada data yang disediakan akan dilakukan evaluasi modelling guna mengetahui seberapa baik model bekerja, mengecek apakah model overfitting atau underfitting, mengetahui kelemahan model, membandingkan model, dan mengukur performa per kelas. Evaluasi dasar dilakukan dengan membandingkan seluruh model untuk mencari model terbaik dengan melihat beberapa aspek pendukungnya. Berikut penjelasan detail yang digunakan untuk mengevaluasi model:

```
model_files = {
    "TF-IDF + Logistic Regression": "/content/model_tfidf_LR.csv",
    "TF-IDF + SVM": "/content/model_tfidf_SVM.csv",
    "TF-IDF + Random Forest": "/content/model_tfidf_RF.csv",

    "Word2Vec + Logistic Regression": "/content/model_W2V_LR.csv",
    "Word2Vec + SVM": "/content/model_tfidf_SVM.csv",
    "Word2Vec + Random Forest": "/content/model_tfidf_RF.csv",

    "FastText + Logistic Regression": "/content/model_predictions_fasttext_LR.csv",
    "FastText + SVM": "/content/model_predictions_fasttext_SVM.csv",
    "FastText + Random Forest": "/content/model_predictions_fasttext_RF.csv",
}
```

Script akan membaca semua model yang sudah dibentuk dengan mendefinisikan nama model dan lokasi file hasil prediksi masing-masing setiap CSV yang memuat kolom `y_true` (label asli) dan `y_pred` (label hasil prediksi model). Dari file model yang dibaca akan mengambil metrik evaluasi dan mengubahnya menjadi dictionary supaya dapat diakses per nilainya. Dibuatkan fungsi untuk menghitung accuracy, macro average, dan weighted average.

```
for model_name, file_path in model_files.items():
    df = pd.read_csv(file_path)

    y_true = df["y_true"]
    y_pred = df["y_pred"]

    metrics = extract_metrics(y_true, y_pred, model_name)
    results.append(metrics)
```

Selanjutnya semua model akan di loop dan diambil metriknya dan disimpan dalam list `results` dan dikonversi menjadi DataFrame supaya rapi dan mudah dianalisis. Berikut output yang dihasilkan dari seluruh proses membandingkan semua model:

===== TABEL PERBANDINGAN 9 MODEL =====						
	Model	Accuracy	Precision_0	Recall_0	F1_0	\
0	TF-IDF + Logistic Regression	0.909574	0.949416	0.865248	0.905380	
1	TF-IDF + SVM	0.939716	0.973282	0.904255	0.937500	
2	TF-IDF + Random Forest	0.957447	0.944828	0.971631	0.958042	
3	Word2Vec + Logistic Regression	0.781915	0.806950	0.741135	0.772643	
4	Word2Vec + SVM	0.939716	0.973282	0.904255	0.937500	
5	Word2Vec + Random Forest	0.957447	0.944828	0.971631	0.958042	
6	FastText + Logistic Regression	0.760638	0.852941	0.822695	0.837545	
7	FastText + SVM	0.789894	0.814241	0.932624	0.869421	
8	FastText + Random Forest	0.784574	0.807339	0.936170	0.866995	
	Support_0	Precision_1	Recall_1	F1_1	Support_1	Macro_Precision \
0	282.0	0.876221	0.953901	0.913413	282.0	0.912819
1	282.0	0.910596	0.975177	0.941781	282.0	0.941939
2	282.0	0.970803	0.943262	0.956835	282.0	0.957815
3	282.0	0.760656	0.822695	0.790460	282.0	0.783803
4	282.0	0.910596	0.975177	0.941781	282.0	0.941939
5	282.0	0.970803	0.943262	0.956835	282.0	0.957815
6	282.0	0.519231	0.574468	0.545455	94.0	0.686086
7	282.0	0.641509	0.361702	0.462585	94.0	0.727875
8	282.0	0.632653	0.329787	0.433566	94.0	0.719996
	Macro_Recall	Macro_F1	Weighted_Precision	Weighted_Recall	Weighted_F1	
0	0.909574	0.909396	0.912819	0.909574	0.909396	
1	0.939716	0.939640	0.941939	0.939716	0.939640	
2	0.957447	0.957438	0.957815	0.957447	0.957438	
3	0.781915	0.781552	0.783803	0.781915	0.781552	
4	0.939716	0.939640	0.941939	0.939716	0.939640	
5	0.957447	0.957438	0.957815	0.957447	0.957438	
6	0.698582	0.691500	0.769514	0.760638	0.764522	
7	0.647163	0.666003	0.771058	0.789894	0.767712	
8	0.632979	0.650281	0.763668	0.784574	0.758638	
File perbandingan disimpan sebagai perbandingan_9_model.csv						

Hasil evaluasi menunjukkan bahwa model terbaik diperoleh dari kombinasi Random Forest dengan representasi TF-IDF maupun Word2Vec, yang keduanya menghasilkan performa identik. Model ini mencapai akurasi sebesar 95,74% dengan nilai Macro F1 dan Weighted F1 sebesar 0,957, menjadikannya model paling konsisten di antara sembilan model yang diuji. Selain memberikan precision dan recall yang seimbang pada kedua kelas, model ini juga mampu menggeneralisasi pola sarkasme dengan sangat baik tanpa menunjukkan gejala overfitting yang signifikan. Dengan demikian, Random Forest terbukti menjadi algoritma paling efektif dalam memanfaatkan representasi teks baik berbasis frekuensi (TF-IDF) maupun berbasis semantik (Word2Vec) untuk tugas klasifikasi sarkasme pada penelitian ini.

2.6.2 Cek Gap Train Vs Test

Sebagai bagian dari evaluasi kualitas model, dilakukan analisis perbedaan performa antara data training dan data testing untuk mendeteksi potensi overfitting. Dengan

menghitung selisih akurasi (gap) antara train dan test pada sembilan kombinasi model dan representasi teks, proses ini membantu mengidentifikasi model yang paling stabil serta memastikan bahwa performa tinggi yang diperoleh bukan hasil dari ketidakseimbangan pembelajaran, melainkan kemampuan model dalam menggeneralisasi pola sarkasme secara konsisten. Berikut penjelasan detail nya:

```
models = {  
    "LR": LogisticRegression(max_iter=2000),  
    "SVM": SVC(),  
    "RandomForest": RandomForestClassifier(random_state=42)  
}
```

Setelah memastikan load data dari 3 representasi modelling dengan tujuan memisahkan fitur X_Train dan y_train juga menyiapkan data untuk setiap representasi agar bisa dipakai model dilanjut dengan inisialisasi 3 algoritma model yang telah diuji, pada inisialisasi Random Forest dipasang state agar ssetiap kali run nilai tidak berubah.

```
def evaluate_model(name, model, X_train, y_train, X_test, y_test, results):  
    model.fit(X_train, y_train)  
  
    train_acc = model.score(X_train, y_train)  
    test_acc = model.score(X_test, y_test)  
  
    results.append({  
        "Model": name,  
        "Train Accuracy": round(train_acc, 4),  
        "Test Accuracy": round(test_acc, 4),  
        "Gap (Overfitting)": round(train_acc - test_acc, 4)  
    })  
    return results
```

Dibuat fungsi untuk menghitung overfitting yang akan melakukan pelatihan model, menghitung akurasi pada train dan test, menghitung selisih (Train – Test), dan menyimpan hasilnya dalam bentuk list results. Gap kecil menandakan model stabil, sedangkan gap besar mendandakan model terjadi overfitting. Berikut output yang dihasilkan dari keseluruhan script:

```
===== HASIL CEK OVERFITTING 9 MODEL =====
```

	Model	Train Accuracy	Test Accuracy	Gap (Overfitting)
5	FastText + RandomForest	1.0000	0.7819	0.2181
4	FastText + SVM	0.8983	0.7793	0.1191
3	FastText + LR	0.8299	0.7580	0.0720
0	TFIDF + LR	0.9756	0.9113	0.0642
8	Word2Vec + RandomForest	1.0000	0.9468	0.0532
7	Word2Vec + SVM	0.8308	0.7926	0.0383
2	TFIDF + RandomForest	1.0000	0.9645	0.0355
1	TFIDF + SVM	0.9991	0.9663	0.0328
6	Word2Vec + LR	0.7948	0.7908	0.0041

```
Laporan disimpan sebagai overfitting report 9 model.csv
```

Hasil evaluasi menunjukkan bahwa tingkat overfitting bervariasi cukup signifikan antar model. Kombinasi berbasis FastText, khususnya FastText + Random Forest, memiliki gap terbesar antara akurasi training dan testing, menandakan overfitting yang kuat. Sebaliknya, model seperti Word2Vec + Logistic Regression memiliki gap paling kecil, menunjukkan generalisasi yang sangat stabil meskipun akurasinya tidak setinggi model lain. Secara keseluruhan, model berbasis TF-IDF dan Word2Vec menunjukkan performa yang lebih konsisten dan tidak mudah overfit, sementara representasi FastText cenderung membuat model belajar terlalu spesifik terhadap data training. Evaluasi gap ini membantu memastikan bahwa performa model yang tinggi tidak berasal dari overfitting, melainkan kemampuan generalisasi yang baik terhadap data baru.

BAB III

KESIMPULAN & SARAN

3.1 Kesimpulan

Project ini cukup berhasil membangun sebuah sistem deteksi komentar sarkasme berbasis machine learning dengan memanfaatkan data hasil scraping dari YouTube dan TikTok, serta dataset sarkas yang diunduh dari HuggingFace sebagai data pelatihan. Proses pengolahan data dilakukan secara komprehensif mulai dari pengumpulan, pembersihan, pre-processing, eksplorasi, hingga pemodelan untuk memperoleh model yang paling efektif.

Dari sisi pengumpulan data, scraping dari dua platform menghasilkan total 2000 komentar yang kemudian diintegrasikan ke dalam satu dataset terstruktur. Komentar dari YouTube diperoleh melalui API resmi, sementara data TikTok dikumpulkan menggunakan Apify Store. Kedua sumber data memberikan informasi kontekstual yang kaya dan relevan untuk keperluan analisis.

Pada tahap cleaning dan pre-processing, data dinormalisasi melalui penghapusan URL, mention, hashtag, karakter noise, serta perbaikan spasi. Selain itu dilakukan juga normalisasi slang, konversi emoji ke token, pemisahan emoji dari teks, serta pengubahan keseluruhan teks menjadi lowercase. Seluruh proses ini menghasilkan kolom `clean_text` yang bersih, konsisten, dan siap digunakan sebagai input untuk ekstraksi fitur serta modelling.

Tahap Exploratory Data Analysis (EDA) memberikan gambaran menyeluruh mengenai struktur dan karakteristik dataset. Analisis meliputi pemeriksaan missing value, distribusi panjang teks, distribusi jumlah kata, serta karakteristik interaksi pengguna seperti `favorite_count`. EDA juga menunjukkan komposisi data per platform serta persentase teks yang mengandung kata kunci terkait topik perselingkuhan. Hasil EDA memastikan bahwa data layak digunakan untuk modelling serta memberikan insight mengenai pola umum dalam komentar pengguna.

Pada bagian modelling, dilakukan eksperimen menggunakan tiga representasi teks: TF-IDF, Word2Vec, dan FastText, yang masing-masing dikombinasikan dengan tiga algoritma machine learning: Logistic Regression, Support Vector Machine (SVM), dan Random Forest. Untuk menangani ketidakseimbangan kelas pada dataset sarkas, diterapkan teknik augmentasi seperti synonym replacement dan random swap. Setiap kombinasi model kemudian dievaluasi menggunakan metrik akurasi, precision, recall, f1-score, dan confusion matrix.

Hasil evaluasi menunjukkan bahwa Random Forest, baik menggunakan TF-IDF maupun Word2Vec, menghasilkan performa terbaik dengan akurasi 95,74% serta nilai Macro dan Weighted F1 sebesar 0,957. Model ini terbukti mampu mengenali pola bahasa sarkasme secara konsisten dan seimbang pada kedua kelas tanpa menunjukkan kecenderungan overfitting. Analisis gap train-test juga memperkuat bahwa model berbasis TF-IDF dan Word2Vec lebih stabil dibandingkan FastText, yang cenderung mengalami overfitting.

Secara keseluruhan, penelitian ini berhasil membangun pipeline deteksi sarkasme yang lengkap mulai dari scraping, pengolahan data, eksplorasi, hingga pemilihan model terbaik.

Kombinasi Random Forest + TF-IDF/Word2Vec dapat direkomendasikan sebagai pendekatan paling efektif untuk tugas klasifikasi sarkasme pada dataset berbahasa Indonesia, terutama teks pendek dari media sosial. Hasil ini menunjukkan bahwa integrasi scraping data real-world dan teknik NLP modern mampu menghasilkan model yang handal dalam memahami pola bahasa sarkastik pada komentar daring.

3.2 Kendala

3.2.1 Kesulitan Menentukan Model Terbaik

Beberapa model seperti SVM dan FastText menunjukkan performa yang tidak stabil, bahkan berpotensi overfitting. Dibutuhkan banyak iterasi, perbandingan metrik, dan pengujian lintas representasi teks (TF-IDF, Word2Vec, FastText) sebelum ditemukan kombinasi model yang paling optimal.

3.2.2 Kendala dalam Preprocessing

Dataset Twitter memiliki struktur teks yang menyertakan elemen-elemen seperti hashtag (#) dan mention (@user) yang melekat pada setiap komentar. Saat dilakukan preprocessing, tag tersebut ikut masuk sebagai bagian teks sehingga perlu dilakukan pembersihan tambahan agar tidak mengganggu proses ekstraksi fitur. Sebaliknya, komentar hasil scraping TikTok tidak menyertakan tag apa pun baik hashtag maupun mention karena platform tersebut otomatis menghilangkannya pada hasil scraping. Ketidakkonsistenan struktur ini menyebabkan tantangan dalam menyelaraskan format dataset, karena preprocessing harus menyesuaikan dua tipe teks yang berbeda untuk menjaga kualitas fitur akhir.

3.2.3 Kendala dalam Penentuan Label Sarkasme

Penentuan label pada dataset sarkas menjadi tantangan karena bahasa sarkasme sulit dikenali secara eksplisit dan sering kali memiliki makna yang ambigu. Banyak komentar menggunakan ironi, sindiran halus, atau konteks percakapan yang tidak lengkap, sehingga model kesulitan membedakan apakah suatu kalimat benar-benar sarkas atau hanya ekspresi bercanda biasa. Kondisi ini menuntut adanya model yang stabil, dengan presisi yang seimbang antara kelas *sarkas* dan *non-sarkas*, agar tidak berat sebelah dan mampu menangkap ciri-ciri sarkasme secara konsisten. Model yang terlalu bias ke kelas mayoritas akan gagal mengenali komentar sarkas, sedangkan model yang terlalu sensitif dapat salah mendeteksi komentar normal sebagai sarkas. Oleh sebab itu, penentuan label yang akurat dan pengembangan model yang tahan terhadap ambiguitas bahasa menjadi salah satu kendala utama dalam penelitian ini.

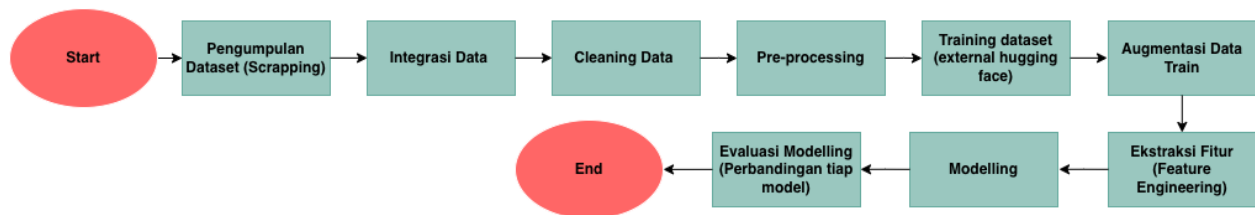
LAMPIRAN

Bany, A. K. N. (2022). *Analisis sentimen dan deteksi emosi dengan pendekatan lexicon pada judul berita media online mengenai COVID-19 di Indonesia* [Skripsi Sarjana, Universitas Islam Negeri Syarif Hidayatullah Jakarta]. UIN Syarif Hidayatullah Jakarta Institutional Repository.

<https://repository.uinjkt.ac.id/dspace/bitstream/123456789/64434/2/AMALIA%20KHAERUNNISA%20NURSYA%20BANY-FST.pdf>

Davidson, T., Warmley, D., Macy, M., & Weber, I. (2017). *Automated hate speech detection and the problem of offensive language*. arXiv. <https://arxiv.org/abs/1703.04009>

Flowchart:



Video presentasi progress: <https://youtu.be/LT2NF30nzhE>

Video Final Project: <https://youtu.be/X7N7c-UT17k>